CODE NOMADS / GROUP 68

CampusConnect

Student Ride System

Campus rides. Clear journeys. Connected support.

PHASE 4 / CONSOLIDATED PROJECT REPORT

Implementation, testing and evaluation

PROJECT INFORMATION

Module: Project (NPRT63030 / NPRT 630)

Lecturer: Melvin Kisten

Original submission date: 12 October 2026

Report revised: 13 October 2026

GROUP MEMBERS

Khumbudzo Munzhelele	202314571
Menzi Sigwebela	202412126
Brilliant Sifunda	202447143
Sanele Dlomo	202447772

RIDER / DRIVER / ADMINISTRATOR / CAMPUS SECURITY

OVERVIEW

Report overview

Executive summary

CampusConnect is a student transport and safety application that connects riders, drivers, university administrators and campus security. It brings booking, trip updates, messaging, identity checks and incident reporting into one organised workflow. The campus context is Sol Plaatje University in Kimberley, with fares displayed in South African rand.

Code Nomads developed the project across four phases: defining the problem, modelling requirements, designing interfaces and data, and implementing and evaluating the system. The current application uses an Expo/React Native client, a Spring Boot backend and MySQL storage. It is a working student-project prototype, with selected functions supported by recorded automated tests.

Two participants tested the application. Zinhle Mahlangu covered the rider role; Zwelibanzi covered driver, administrator and campus security roles. Their feedback informed account recovery and driver approval improvements. This report includes approximate session times, observed difficulties and Zinhle's 3/5 ease rating, without inventing missing scores.

Contents

Section	Pages
Context and requirements: Phases 1–2	3–6
Interface and data design: Phase 3	7–10
Architecture and implementation: Phase 4	11–14
Automated and user testing; applied improvements	15–18
Demonstration, reflection, conclusion and sources	19–20

Evidence is identified as current code, recorded test output, participant feedback or earlier design material. Recorded tests were reviewed for this revision; they were not rerun during document preparation. The original report lists 12 October 2026 as the submission date. This revised document was prepared on 13 October 2026.

PHASE 1

The problem and proposed solution

Campus transport needs

Students may need transport between lectures, residences, libraries and off-campus accommodation. The original proposal focuses on arranging rides, knowing who the driver is and finding help when a journey raises a safety concern. These are the project's motivating needs, rather than findings from a campus-wide survey.

Informal calls and messaging groups can coordinate a pickup, but the request, driver decision and journey details may sit in separate conversations. CampusConnect provides one record of the journey and dedicated tasks for each role. Its purpose is to make coordination easier to follow and give campus staff a structured way to review drivers and incidents.

User	Need addressed
Rider	Request transport, understand the fare, follow trip progress and report a concern.
Driver	Submit an application, know the approval outcome, accept requests and manage a trip.
Administrator	Review applications, inspect documents and oversee users, rides and settings.
Campus security	Review active rides, SOS details, location and available evidence.

How the proposed solution responds

The rider selects a pickup and destination, reviews the fare estimate and requests a ride. An eligible driver accepts the request and advances the trip through its stages. Both parties can use ride-specific communication and provide feedback after completion. Administrator and security interfaces support the related approval and incident workflows.

Scope of the comparison

The earlier proposal names general ride-hailing services and WhatsApp groups as alternatives. This report treats that comparison as project context, not a verified review of current market prices or platform capabilities. No measured cost saving, campus adoption rate or guaranteed safety outcome is claimed. Payment records and fare displays also do not establish real payment settlement.

PHASE 2

Requirements and quality goals

Requirements connect a user need with an action and an expected response. They also guide what should be checked during testing. The following requirements reflect the current role-based workflows.

ID / area	Required behaviour
FR01 — Accounts	Create an account, sign in by role and recover a forgotten password.
FR02 — Identity	Enroll a face reference and require fresh confirmation for rider requests and driver acceptance.
FR03 — Driver approval	Submit application details; allow an administrator to decide; display the result to the driver.
FR04 — Ride coordination	Request or schedule transport; accept an eligible request; show trip status and history.
FR05 — Communication	Allow ride participants to exchange messages and rate eligible completed rides.
FR06 — Safety	Record an SOS report with available location and audio evidence for security review.
FR07 — Oversight	Support user management, dashboard information, application documents and university settings.

Quality goals

Quality	How it is assessed
Ease of use	Task observations, errors, participant feedback and a clearly defined ease scale.
Reliability	Valid state changes, failed-request handling and repeatable software checks.
Security	Authentication, role restrictions, ownership rules and controlled invalid-action tests.
Performance	Timed request delivery and repeated measurements under known conditions.
Portability and maintainability	Responsive screens, physical-device checks and separated client, service and data code.

These goals are not service guarantees. For example, the observed 15–20 second request delay is an informal timing from testing, not an established average or promised response time. Native-device behaviour, repeat performance measurements and further user testing remain necessary.

PHASE 2

Role workflows and system boundary

Each role has a distinct responsibility. Riders request journeys, drivers operate them, administrators decide whether driver applications are approved, and security staff review reported incidents. The diagram summarises responsibilities in the current project.

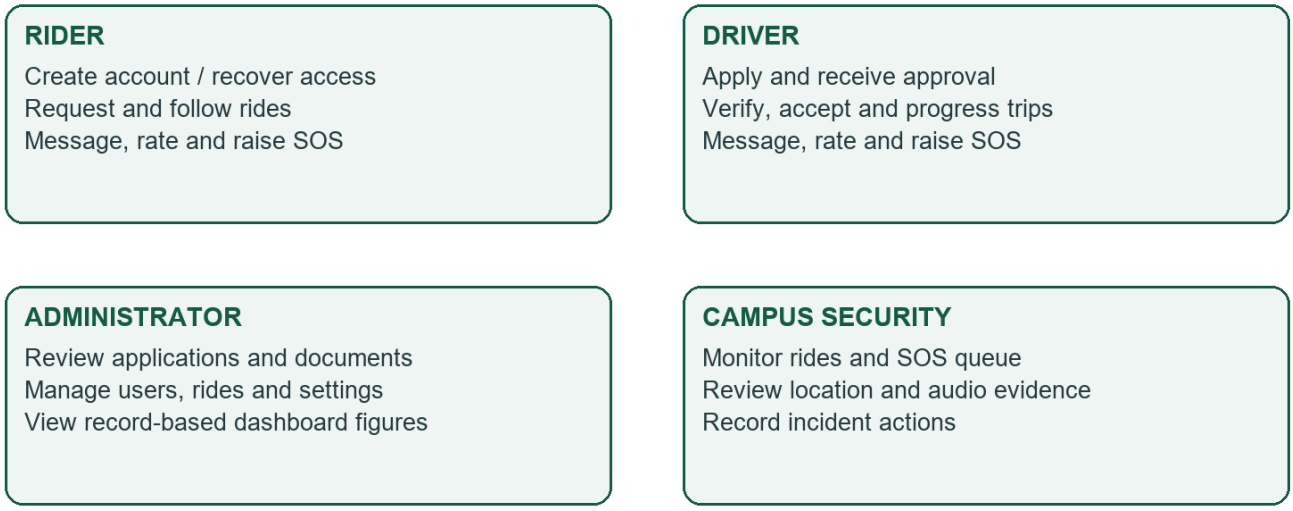


Figure 1. Role responsibilities redrawn for the current application. This is a workflow overview, not a claim that every path has been tested.

Inputs, processing and outputs

Process	Input and processing	Visible result
Registration	Details, role and face reference; validate and save.	Account creation or an actionable error.
Driver approval	Application and documents; administrator decision.	Pending or approved status shown to the driver.
Booking	Locations and fresh face confirmation; save the request.	Pending ride and subsequent driver response.
Trip operation	Assigned driver advances the journey.	Current status, messages and trip record.
SOS review	Alert, available GPS and audio; security review.	Incident information and recorded actions.

Role-specific screens make tasks easier to find, but backend checks remain essential. A hidden button alone does not prevent an unauthorised request; the API must verify who is acting and whether they may act on that record.

PHASE 2

Trip progression and interaction

A ride is represented by an explicit status so that both participants can understand its progress. The main accepted-trip sequence is shown below. Scheduled and cancelled rides are related paths and should be tested independently.

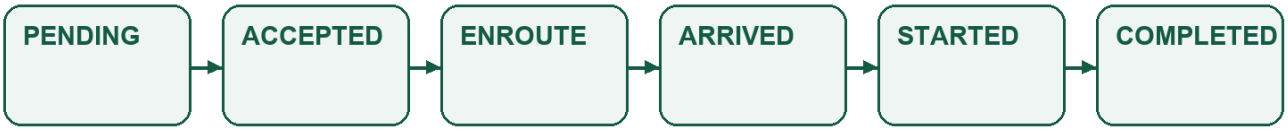


Figure 2. Main ride lifecycle. Labels are application states; the diagram does not assert that every transition has independent test coverage.

Booking and acceptance sequence

Step	Interaction
1. Prepare	Rider enters pickup and destination and reviews the journey information.
2. Verify	A fresh rider image is compared with the enrolled reference; the backend issues a short-lived confirmation.
3. Request	The request uses that confirmation and is stored for eligible drivers.
4. Review	Driver sees the request and completes the required fresh face confirmation.
5. Accept	The service checks eligibility and current assignment before accepting.
6. Operate	Driver progresses the ride; rider receives updated trip information.
7. Finish	Completion enables eligible ratings and updates trip history.

Handling interrupted actions

A face mismatch or unavailable matching service must not silently approve a ride action. If another driver has already claimed the ride, the acceptance service rejects the competing assignment. A repeated acceptance by the already assigned, verified driver returns the existing result in the tested service path.

These checks help preserve the meaning of each stage. Simultaneous real requests, network interruptions and the complete journey across physical devices still need integrated testing. The interface should show the current state and a clear recovery action when an operation fails.

PHASE 3

Rider interface and navigation

The rider interface brings account access and journey planning together. New users need a visible route to account creation; returning users need sign-in and account recovery. Booking then centres on selecting locations, understanding the estimate and confirming the request.

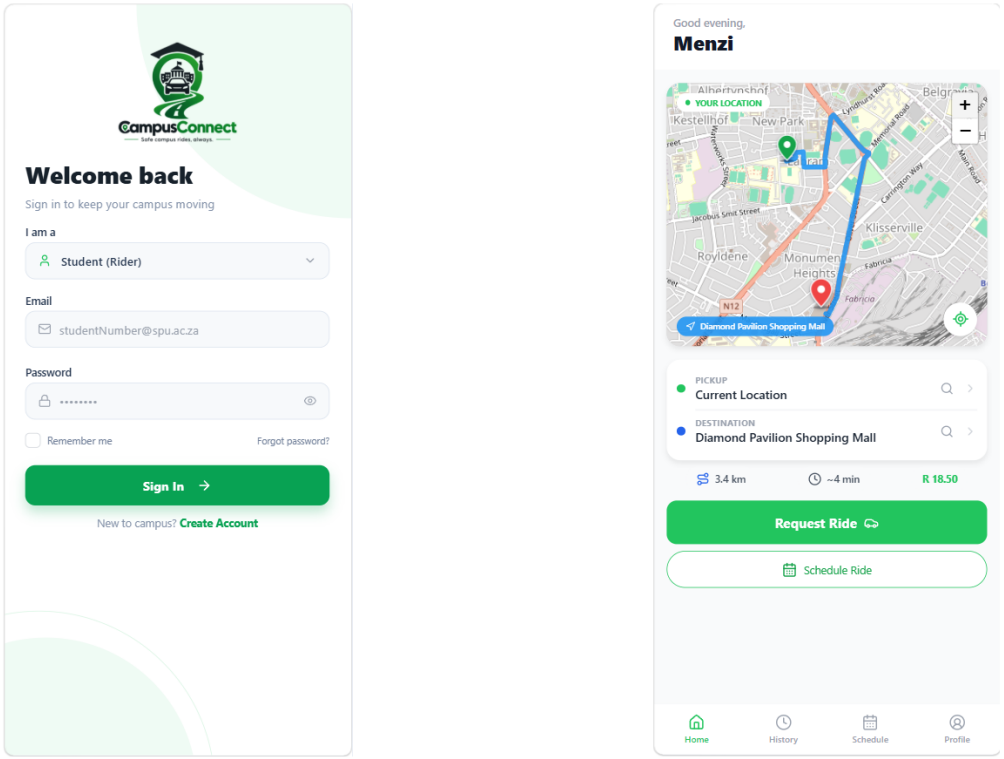


Figure 3. Project-supplied sign-in and rider booking captures. They illustrate interface states; displayed names, fares and map details are not measured service outcomes.

Design decisions and testing feedback

Zinhle initially selected “Sign in” before creating an account. She noticed the mistake and found the “Create account” link. This shows that recovery was possible, while the first step could be clearer. Her overall ease rating was 3/5, and she requested password reset in case she forgot her password.

The current sign-in screen includes “Forgot password?”. Documented interface refinements also include shared responsive layouts, persistent rider navigation, clearer back controls and scrollable dialogs. These changes aim to keep important actions visible on smaller screens. A new participant rating has not been supplied to measure their effect.

PHASE 3

Driver, admin and security interfaces

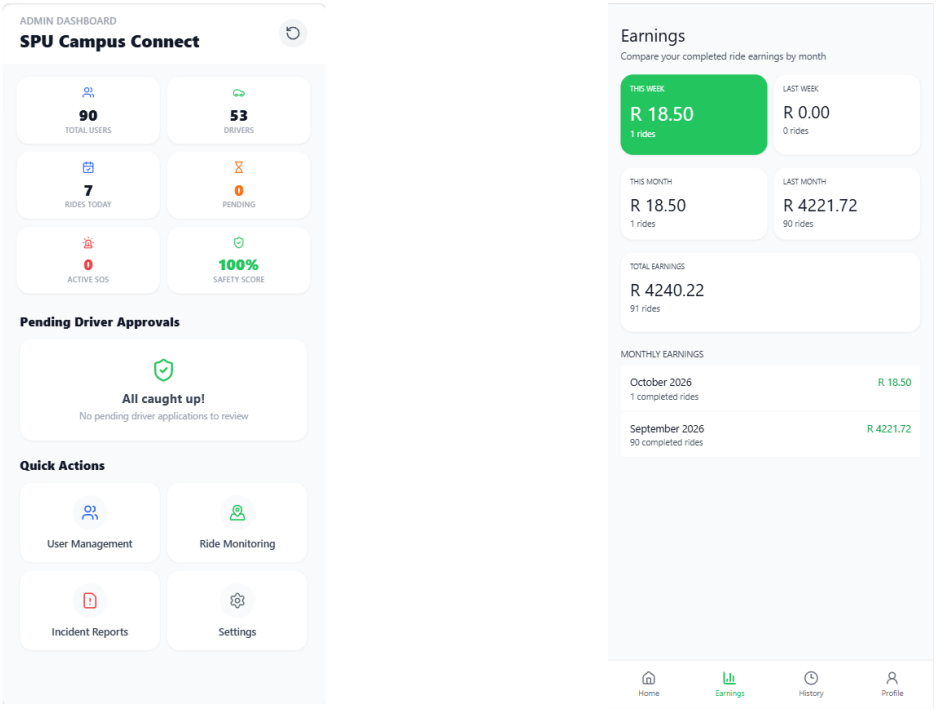


Figure 4. Project-supplied administrator dashboard and driver earnings captures. Statistics and amounts in these images are illustrative screen content.

Driver approval feedback

Zwelibanzi encountered a “Check status” button that did not work. The current flow removes that button and refreshes approval status automatically while the driver is pending. Once approved, the screen displays “Account Approved” and directs the driver to log in to continue. This is confirmed by the current DriverDashboardScreen code and the project owner.

Administration and security

Administrators review driver applications and available documents, make decisions and access users, rides and university settings. Security staff review active rides and SOS incidents with available location and audio evidence. The interfaces organise different responsibilities around the same journey records.

The driver must see a meaningful result after a decision, while administrators must see accurate data or a clear loading failure. Security users need understandable incident details and visible actions. An on-screen incident update records a system action; it is not evidence of a real-world emergency response.

PHASE 3

Database structure and relationships

The application uses a relational database to retain accounts, driver profiles, trips and associated records. Keeping a journey's messages and alerts connected to that journey supports retrieval and permission checks. The diagram shows the central logical relationships used in the application.

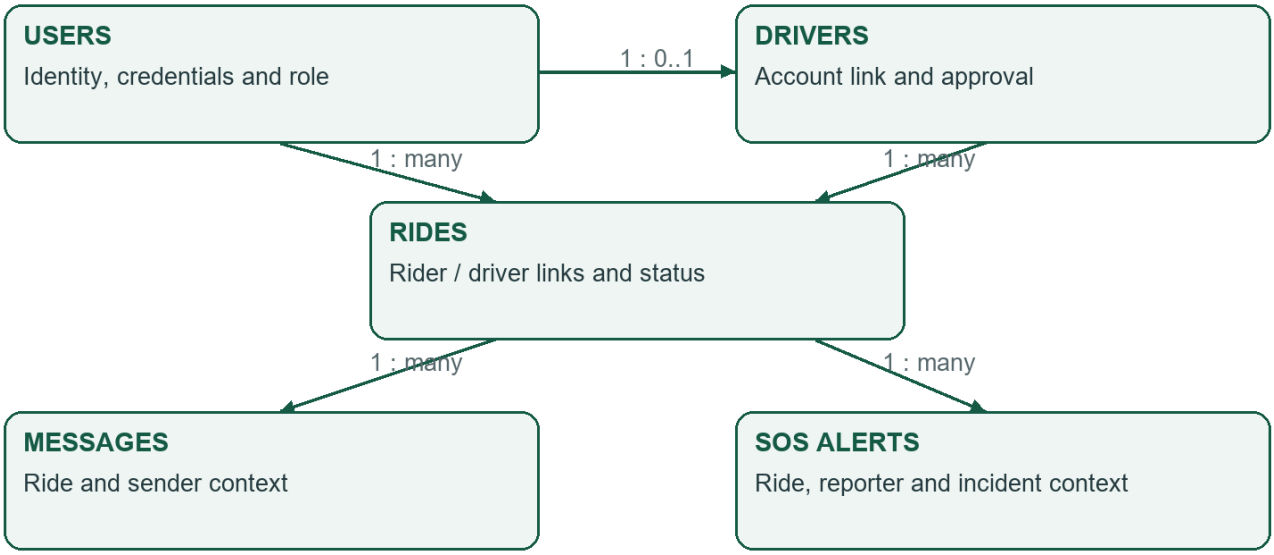


Figure 5. Simplified logical data model, redrawn from current entities. Lines show application associations and do not certify live database foreign-key constraints.

Relationship rationale

A user account stores common identity and account information. A driver record stores driver-specific details and approval status. A ride refers to its rider and, after acceptance, its driver. Messages have a ride context and sender; SOS alerts retain the journey context and the person who raised the concern.

Password-reset codes are associated with an email address and expire. University settings are managed separately from individual trips. Payment-related records support display and preference features; they should not be described as a completed payment-processing integration.

Schema verification

Entity mappings and SQL files describe the intended data structure, but they are not a substitute for a clean-install check. The project includes migration and schema-maintenance material; the installed schema must be checked for consistency before deployment. A live schema export was not collected during this report revision.

Selected data dictionary and validation

The selected fields below explain how key records support the workflows. Types come from the current entity mappings. This is a concise dictionary rather than a complete live database specification.

Entity / field	Mapped type	Meaning / constraint
users.user_id	Long / BIGINT	Generated account identifier.
users.email	String	Unique, required account email in the entity mapping.
users.password_hash	String	Required hashed password; excluded from JSON through the entity annotation.
users.role	Enum stored as text	Determines role-specific account behaviour.
drivers.driver_id / user_id	Long / BIGINT	Driver identifier and required link to an account.
drivers.approval_status	Enum stored as text	Tracks the application decision.
rides.ride_id / rider_id	Long / BIGINT	Journey identifier and rider reference.
rides.driver_id	Long / BIGINT	Unassigned until a driver accepts.
rides.fare	BigDecimal	Stored fare amount; not proof that funds were settled.
rides.status	Enum stored as text	Current journey state.
rides.scheduled_at	LocalDateTime	Scheduled journey time, where applicable.
rides.rider_rating_comment	String, length 500	Optional comment associated with rating.
messages.message_text	String	Ride message; endpoint rejects text longer than 1,000 characters.

Validation belongs at more than one layer

The interface can help users enter valid information, but the backend also needs to reject invalid requests. Record ownership, role permissions and state checks depend on the authenticated account. Entity constraints describe storage requirements; service and controller rules describe whether an action is permitted.

The full project dictionary is retained in docs/report/data-dictionary.csv. Some entries reflect an earlier code review, so it should be reconciled with current mappings and the installed schema before being used as a deployment specification.

PHASE 4

Architecture and technology choices

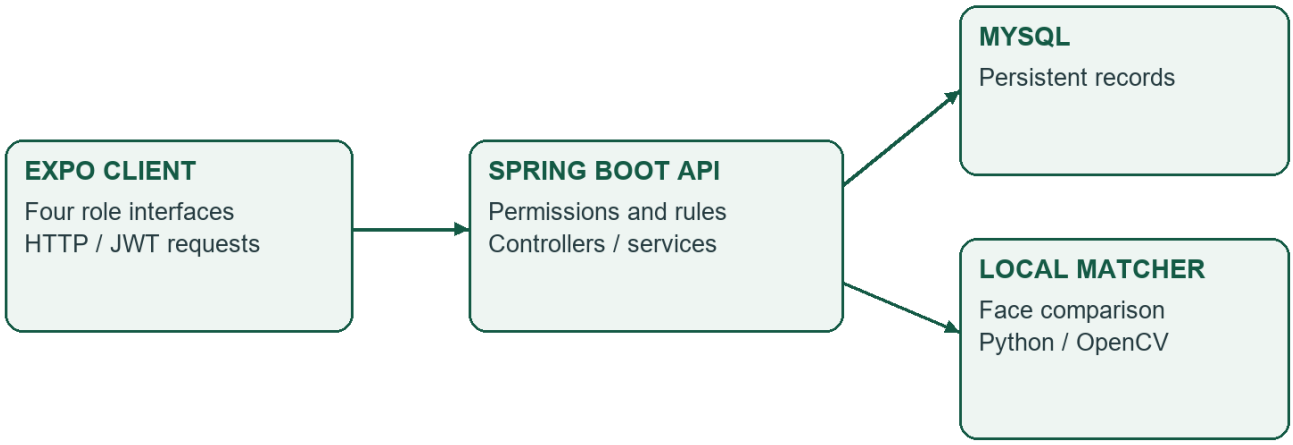


Figure 6. Current system boundaries. The client polls the API for updates; the face-matching helper is a separate local service.

Layer	Technology and purpose
Client	Expo 57, React Native 0.86 and TypeScript 6: role screens and shared interface components.
Navigation and requests	React Navigation, AsyncStorage and Axios: navigation, local state and HTTP requests.
Maps	Leaflet / React Leaflet, React Native Maps, OpenStreetMap and routing support.
Backend	Spring Boot 3.5.0, Java 25 and Maven: controllers, services and validation.
Authentication	Spring Security, JWT and BCrypt: authenticated API access and password hashing.
Persistence	MySQL, Spring Data JPA and JdbcTemplate: account and journey records.
Face comparison	Local Python/OpenCV helper using YuNet and SFace models.
Verification	JUnit, Mockito, MockMvc and browser scripts for selected behaviours.

Why the layers are separate

The client presents actions, the backend decides whether they are permitted, and the database retains the resulting records. Separating these responsibilities makes a screen change easier to distinguish from a business-rule change. The matching helper adds an external dependency to face confirmation; the backend reports a failure if that helper cannot complete the comparison. The repository is organised into mobile/, backend/, database/mysql/ and docs/. Dependency versions in this report come from the project manifests rather than assumptions about vendor releases.

PHASE 4

Accounts, recovery and identity checks

Authentication and account recovery

Account sign-in uses password checking and a JWT bearer token for API requests. BCrypt is used for password hashing. The configured token expiry is distinct from a face-confirmation receipt: signing in does not by itself satisfy the fresh face requirement for a ride action.

The “Forgot password?” route supports Zinhle’s request for account recovery. The service generates a six-digit code, stores a hash, gives it a 10-minute expiry and limits verification attempts to five. It also limits rapid repeat requests. Successful recovery replaces the password; actual inbox delivery depends on the configured email service.

Face enrollment and fresh confirmation

Rider and driver registration capture a reference face image. Before a rider request or driver acceptance, a fresh image is compared with the relevant account’s reference. The local helper detects a single clear face and compares face features. It is a matching workflow, not an independent proof of legal identity.

Control in current code	Purpose
Reference required	Accounts without an enrolled reference cannot proceed through the fresh matching path.
Matching-service result required	A mismatch or unavailable service returns an error rather than confirmation.
120-second receipt lifetime	Limits how long a successful comparison can authorise a ride action.
Account and action binding	The receipt must belong to the acting user and the corresponding request details.
Single-use receipt	Consumption removes the receipt so it cannot be reused for another request.

What still needs evaluation

Code checks do not establish biometric accuracy across all users, lighting or devices. The project needs repeat tests for poor lighting, denied camera permissions, mismatches, retries and service downtime. Consent, retention and access to enrolled images also require a documented deployment policy. No liveness or anti-spoofing guarantee is claimed.

PHASE 4

Ride operations, messages and ratings

Request and acceptance

A rider prepares journey details and completes fresh face confirmation before submitting a request. An approved driver reviews the available request and completes driver confirmation before acceptance. The acceptance service checks the driver's role, approval and the current state of the ride before assigning it.

Service tests cover a failed confirmation leaving a ride unassigned, a successful acceptance recording the driver, a competing driver having already claimed the ride, and a rider attempting to accept as a driver. These are controlled service checks rather than a load test of simultaneous production requests.

Trip updates and communication

The driver progresses the accepted trip through its expected stages. The client refreshes trip and message information through polling. This approach is straightforward for the prototype, although the observed 15–20 second request delivery delay makes refresh behaviour an important area for repeat measurement.

Function	Business meaning	Verification concern
Journey status	Shows where the ride is in its lifecycle.	Valid transitions, cancellation and reconnection.
Ride messaging	Keeps communication linked to the relevant ride.	Participant access and length validation.
Ratings	Collects feedback on eligible completed journeys.	Rating limits, ownership and duplicate submission.
History	Retains completed and other journey records.	Correct account scope and consistent displayed data.
Fare and earnings displays	Shows estimates and totals from journey records.	Consistent rand formatting and clear payment boundaries.

Communicating failure

A user needs to know whether a request was sent, whether it is awaiting a driver and whether an error needs another attempt. A network or database failure should not be displayed as a successful empty result. The report does not claim that request delivery speed has improved; a controlled set of timed attempts is needed to establish that.

PHASE 4

Administration and campus safety

Driver approval and document review

The administrator reviews pending applications and supporting information before approving or rejecting a driver. The driver's pending screen polls for an approval update and shows the outcome, replacing the failed manual "Check status" action reported by Zwelibanzi. Driver access and ride acceptance remain subject to backend rules.

Document checks verify that a stored document can be returned, an unknown document kind is rejected, and document metadata cannot be supplied through public JSON registration. These checks establish selected handling rules; they are not a complete document-security audit.

Dashboard reliability

The administrator dashboard calculates figures from records rather than presenting fixed reassuring values. Tests cover active SOS and completed-ride counts, the absence of a percentage when no completed rides exist, South African date boundaries and database failure responses. A failed approval query must not be presented as an empty queue.

SOS reporting and security review

An assigned driver can raise an SOS concern with journey context and available location and audio evidence. Security staff can review the alert and record incident actions. The recorded workflow check verifies that the alert reaches the security queue and that an unrelated driver cannot report or attach evidence to that ride.

Safety feature	Evidence boundary
SOS queue and incident records	Demonstrates in-system reporting and review, not guaranteed campus response.
Location context	Depends on permission and available device location data.
Audio evidence	Controller limit is 2 MB; native microphone behaviour needs device validation.
Recorded dispatch or resolution	A stored action is not proof of real-world attendance or response time.

Zwelibanzi tested the campus security role as well as driver and administrator tasks. That coverage is user testing of the role interface, not a penetration test or a comprehensive cybersecurity assessment.

Recorded software testing

Usability testing asks whether people can understand and complete a workflow. Unit checks exercise a small component under controlled conditions; integration checks exercise cooperating parts of the system. Browser checks can verify layout and actions while substituting API responses. These forms of evidence answer different questions.

Recorded run results

Run / date	Checks	Failures / errors / skipped
Backend run, 12 Oct 2026 at 20:35 SAST	45	0 / 0 / 0
Separate application-document run, 12 Oct 2026 at 20:36 SAST	3	0 / 0 / 0

The 45-check run is recorded in backend/.admin-dashboard-test.log. The separate 3-check run is recorded in backend/.admin-document-test.log. Both report BUILD SUCCESS. The totals are kept separate because the coverage can overlap; they are not added into a claim of 48 distinct checks.

Selected areas checked

Area	Checks represented in project tests
Accounts and recovery	Authentication behaviour and password-reset conditions.
Face confirmation and acceptance	Receipt rules, rider request guards and driver assignment conditions.
Administrator operations	Application decisions, dashboard calculations and failure handling.
Documents	Stored document retrieval and selected invalid input handling.
SOS workflow	Assigned-driver reporting, security access and rejection of unrelated-driver actions.

Frontend evidence and limits

Project testing notes record a successful TypeScript check on 5 October 2026 and browser scripts for responsive layouts, admin approvals, ratings and driver SOS. Several browser checks intercept API calls and use controlled responses, so their success does not prove a live end-to-end journey or participant satisfaction.

This revision corrects the supplied report’s older 29-test summary using recorded later runs. Tests were not rerun to prepare the report. Physical-device camera, microphone and location permissions, email delivery and complete live journeys remain separate validation tasks.

User testing: participants and metrics

Two users tested the application: Zinhle Mahlangu as a rider and Zwelibanzi across driver, administrator and campus security roles. The project owner supplied the following observations. The participant test date, devices, build identifier and stopwatch logs were not supplied.

Measure	Recorded observation	Interpretation
Zinhle's rider session	About 5 minutes	Approximate overall testing time.
Zwelibanzi's combined session	About 7 minutes	Covers his roles together; not 7 minutes per role.
Feature-time estimate	About 1 minute 30 seconds minimum	General estimate without a task log; not a measured average.
Request delivery to driver	About 15–20 seconds	Time until the request appeared; not driver acceptance time.
Zinhle's overall ease rating	3 out of 5	Reported ease score; no post-change rating supplied.
Zwelibanzi's ease rating	Not supplied	Excluded from satisfaction calculations.

Observed errors and difficulties

Zinhle selected "Sign in" before creating an account. She recognised the mistake and found the "Create account" link. This is one reported navigation mistake with recovery. Zwelibanzi selected "Check status", but the button did not work. This is one reported functional failure, rather than evidence that he used the control incorrectly.

How the measures are interpreted

Task completion times are only partially recorded: the notes provide session estimates and request delivery timing, rather than a timing for each assigned task. The general feature-time estimate cannot be added across an assumed task count because it does not reconcile clearly with the short overall sessions.

An error rate requires the number of errors divided by a defined number of attempts. Total attempts were not supplied, so the two observed difficulties are reported as counts, not a percentage. Ease is assessed on a 1–5 scale, where 1 is very difficult and 5 is very easy. Zinhle's score provides one participant measure; it is not a combined average or a complete satisfaction survey.

Feedback and improvements applied

The improvement register links observed needs with the current response. It distinguishes changes confirmed by the project owner or code from recommendations still awaiting validation.

Feedback / need	Applied response	Evidence and remaining check
Zinhle wanted recovery if she forgot her password.	“Forgot password?” is available; the backend supports reset codes and replacement passwords.	Current screen and service code. Repeat the complete flow with a test inbox and user.
Zwelibanzi’s “Check status” button did not work.	The button was removed. Approval updates automatically and “Account Approved” appears after the administrator decision.	Owner confirmation and DriverDashboardScreen code. Repeat the workflow with the participant.
Clearer identity checkpoints were required.	Face reference capture and fresh confirmation support rider requests and driver acceptance.	Current matching and receipt code. Evaluate accuracy and device failures.
Admin figures need reliable records.	Dashboard checks cover record-based counts, unavailable percentages and failed requests.	Recorded automated checks. Confirm against an integrated demonstration database.
Admins need supporting documents.	Application documents can be previewed through the administrator workflow.	Document checks passed; extend permission and invalid-file checks.
Safety reporting must include drivers.	Assigned-driver SOS and evidence reach the security workflow.	Controlled workflow test; native-device recording remains to be checked.

Interface refinements

Project notes also document responsive layouts, persistent rider navigation, clearer back controls and scrollable dialogs. These changes aim to keep the interface understandable and important actions accessible across screen sizes. They are current refinements, but no improved participant score is attributed to them without a retest.

Zinhle’s initial account-entry mistake still suggests a need for clearer first-use guidance. The 15–20 second request delay remains an observed issue for repeat timing; no reduction has been confirmed.

Do results show system improvement?

Evidence-based judgement

Yes: the evidence shows practical improvements in functionality and feedback. Account recovery addresses Zinhle's stated need. Automatic driver approval feedback replaces the failed manual status action encountered by Zwelibanzi. Face confirmation, application-document access, dashboard reliability checks and driver SOS support extend the system's coverage.

The user feedback establishes why the changes matter, and current code and recorded checks establish selected implementation behaviour. The successful 45-check backend run and separate 3-check document run support that judgement under their test conditions.

What the results do not establish

There is no timed before-and-after user study, no post-change ease score and no complete task-attempt record. The report therefore does not claim a percentage reduction in errors, faster task completion or increased satisfaction. Zinhle's 3/5 remains the supplied session rating. A follow-up study is needed to measure the size of the usability improvement.

Next validation	Purpose
Repeat the same role tasks with both participants.	Record task start/end times, success, assistance and errors.
Collect a new ease rating using the same 1–5 scale.	Compare each participant's experience before and after changes.
Repeat admin approval and driver-screen refresh.	Confirm the failed manual action has a reliable replacement.
Time several rider requests under known conditions.	Establish delivery range and investigate the observed delay.
Validate physical-device permissions and face matching.	Check camera, audio, location, retries and unavailable dependencies.

Limits of the study

Two users provide useful early feedback, but they cannot represent all campus users. Approximate timings are affected by what was covered and how sessions were conducted. Recorded code tests do not establish accessibility, biometric accuracy, payment settlement, deployment availability or actual emergency response. These limits guide the next evaluation rather than negate the implemented improvements.

Demonstration and development reflection

Demonstration sequence

The demonstration should present the actual running application with test accounts. The supplied report describes a recorded presentation, but this revision did not inspect a recording or verify that a supervisor question-and-answer session occurred.

Stage	What to demonstrate
1. Explain the need	Campus context, four roles and the project’s scope.
2. Account journey	Registration, sign-in, face enrollment and password recovery.
3. Admin decision	Pending application, documents, approval and automatic driver feedback.
4. Ride journey	Rider request, driver face confirmation, acceptance, statuses and messages.
5. Completion	History and eligible ratings; explain fare and payment boundaries.
6. Safety	Controlled SOS report, available evidence and security review.
7. Evaluation	Recorded test output, participant observations, changes and limitations.

Running the prototype

Start the backend with the project’s Maven setup and configured MySQL database, then start the Expo client from mobile/. The API configuration and a reachable network address must match the browser, emulator or phone. Face matching requires the local helper and its models; password recovery requires configured email delivery. Use the project runbook for exact setup and keep credentials outside submission files.

Lessons supported by the evaluation

The account-entry mistake shows why a visible alternative is useful when a user chooses the wrong first action. The failed status button shows that a control must produce understandable feedback; automatic approval updates simplify that interaction. The request delay shows why testing must include time and system response, not only whether a button can be pressed. Development also requires consistency between interface labels, backend rules and stored data. Separate tests help locate faults, while a complete journey checks how the parts cooperate. These are project lessons drawn from the evidence, rather than invented individual contribution statements.

Conclusion and source record

Overall conclusion

CampusConnect provides an organised campus transport and safety prototype across rider, driver, administrator and campus security roles. The project connects requirements, interface design, data modelling and implementation with recorded software checks and user feedback. Zinhle's rider testing took about 5 minutes and her overall ease rating was 3/5. Zwelibanzi tested his combined roles for about 7 minutes and encountered a failed manual status action. Password recovery and automatic approval feedback now address the stated needs. Further changes strengthen identity checkpoints, administrator information and driver safety reporting.

The evidence supports practical system improvement, while follow-up user testing is needed to quantify changes in task speed, errors and ease. The application remains a student-project prototype, with physical-device validation, integrated deployment checks and live service behaviour requiring further work.

Sources used in this revision

Source	Use in the report
[1] Supplied Phase 4 system report (PDF).	Group identity, lecturer, original submission date, phase structure and earlier narrative.
[2] Project-owner testing feedback.	Participant coverage, approximate timings, two observed difficulties, 3/5 ease rating and confirmed changes.
[3] Current project manifests and source code.	Technology versions, approval screen, password recovery, face matching and selected business rules.
[4] Recorded admin-dashboard and document test logs, 12 October 2026.	Recorded 45-check and separate 3-check successful runs.
[5] Backend tests and project testing notes.	Test scope and limits, frontend evidence and controlled SOS checks.
[6] Project-provided screen captures, entities and dictionary.	Screen illustrations and selected logical data design.

The original report also refers to the group's Phase 1, Phase 2 and Phase 3 submissions, the NPRT630 assignment brief and a technical reference dated 12 October 2026. They are retained as background references; they were not separately revalidated for this revision. Source code and recorded evidence were reviewed on 13 October 2026.